# spanbind: A Simple CI Tool for Detecting Ungrounded Claims in LLM Outputs

**Abinesh Haridoss**
University of North Texas
EXL, Irving, TX
abinesha312@gmail.com

## Abstract

We present `spanbind`, a pytest plugin and command-line tool that fails continuous integration checks when LLM or RAG system outputs contain sentences not bound to verifiable source spans. The tool maps each output sentence to character offsets in a source document, along with a hash of the full document for integrity checking. The default binder uses a heuristic token-overlap approach requiring no external API calls. We provide a simple demonstration on a toy corpus and discuss the tool's current limitations. The system is released as open-source software at `https://github.com/abinesha312/spanbind` and can be installed via pip from the repository.

## 1 Introduction

Large language models (LLMs) and retrieval-augmented generation (RAG) systems frequently produce outputs that mix grounded facts with hallucinations. While substantial research addresses attribution and citation in these systems, practitioners need lightweight tools to catch ungrounded claims during development and testing.

`spanbind` addresses this gap by providing a simple pytest plugin that fails tests when LLM outputs contain sentences that cannot be bound to source documents. The tool is designed for continuous integration (CI) workflows rather than end-user verification, enabling developers to catch grounding failures early.

**Key contributions:** (1) A zero-dependency Python library for binding answer sentences to source spans with document integrity checking via SHA-256 hashes; (2) A pytest plugin enabling assertion-based testing of LLM outputs; (3) A working demonstration on a small corpus with reproducible test results.

**Non-contributions:** This is not a published attribution method, benchmark, or evaluation of any model. The default binder is a simple heuristic. We make no accuracy claims beyond our reproducible test cases.

## 2 System Description

### 2.1 Core API

The core function `bind(answer, sources)` takes an answer string and a list of source documents, returning a list of `BoundClaim` or `UnboundClaim` objects. Each bound claim contains:

- **sentence**: The claim text

- **span**: A `SourceSpan` with `doc_id`, `start`/`end` offsets, `sha256` hash of the full document text, and an `excerpt`
- **score**: The binding confidence (heuristic overlap fraction)
- **binder**: The binding method used

Unbound claims include the sentence, a `reason` (e.g., "below-overlap-threshold"), the best score found, and the binder name.

### 2.2 Heuristic Binder

The default binder performs sentence splitting followed by token-overlap matching:

1. Tokenize the sentence and source documents, removing stopwords
2. For each sentence, slide a variable-width window across each document's tokens
3. Compute coverage: score $= \frac{|\text{sentence tokens} \cap \text{window tokens}|}{|\text{sentence tokens}|}$
4. If any digit in the sentence is absent from the window, set score to 0 (reject wrong numbers)
5. Accept windows with score $\geq 0.55$ (default threshold)
6. Map the best window's token offsets back to character offsets in the source

The binder records the full document's SHA-256 hash, not just the excerpt, ensuring document substitution is detectable.

### 2.3 Integration Modes

**pytest plugin:** The `assert_bound(answer, sources)` function raises `UnboundClaimError` (an `AssertionError` subclass) when any sentence fails to bind, causing test failure.

**CLI:** `spanbind check --answer path/to/answer.txt --source path/to/sources_dir` exits with code 1 if unbound claims exist. The `--json` flag provides machine-readable output.

**Library:** Direct use of `bind()` for programmatic access to binding results.

### 2.4 Optional LLM Binder

An optional LLM-based binder calls a chat completions API (when `SPANBIND_LLM_API_KEY` is set) to generate bindings. This mode prompts the model to return document IDs and offsets, then validates them. No API key is required for normal use; the heuristic binder has zero external dependencies.

## 3 Demonstration and Empirical Results

We report results from running `spanbind` on its own test suite and built-in demo corpus (committed to the repository on August 24, 2026). All numbers are reproducible by running `pytest` and `spanbind demo` after installing from the repository.

### 3.1 Test Suite Results

The repository includes 23 unit and integration tests covering binding correctness, CLI behavior, and plugin integration. All tests pass:

| Test Category | Pass Rate |
|---|---|
| Binding logic (12 tests) | 12/12 |
| CLI behavior (7 tests) | 7/7 |
| pytest plugin (2 tests) | 2/2 |
| Sentence splitting (2 tests) | 2/2 |
| **Total** | **23/23 (100%)** |

### 3.2 Built-in Demo Corpus

Demo corpus: 2 documents (427 chars total) covering support hours and refund policy. Two test answers: (1) *Bound answer* (2 sentences): both bind successfully (scores $> 0.55$); (2) *Unbound answer* (2 sentences): one binds, but "Customers receive a lifetime warranty" is marked unbound (score 0.00), correctly detecting the fabricated warranty claim. Demo reports: `2 bound, 0 unbound` vs. `1 bound, 1 unbound`.

### 3.3 Keyword-Based Fake Generator Test

A non-LLM generator (`fake_generate`) copies sentences sharing keywords with the question. On "When are refunds available and when is the support desk staffed?", it produces 8 sentences. Result: All 8 bind successfully, confirming copy-paste-style RAG answers bind correctly without hallucinations.

### 3.4 Limitations of Current Evaluation

- **Corpus size**: The demo uses 2 documents totaling 427 characters. This is sufficient only for demonstrating the tool's basic functionality.
- **No accuracy benchmark**: We do not measure precision, recall, or F1 against labeled data. The heuristic overlap threshold (0.55) is not empirically validated.
- **No external users**: The tool has 0 GitHub stars and no known production deployments as of August 24, 2026.
- **Paraphrase detection**: The token-overlap heuristic fails on heavy paraphrases (e.g., "Paris" vs. "the City of Light").
- **Number format variants**: "2026" vs. "two thousand twenty-six" may not match despite semantic equivalence.
- **Sentence splitting edge cases**: Bullets, headings, and missing punctuation can cause incorrect segmentation.

## 4 Related Work

**Attribution in RAG:** Substantial research addresses citation and attribution in retrieval-augmented systems, including learned attribution models, natural language inference (NLI) approaches, and span-grounded QA benchmarks. `spanbind` does not implement any specific published method; it is a simple overlap-based tool for CI integration.

**Hallucination detection:** Recent work on LLM hallucination detection includes uncertainty estimation, consistency checking, and external knowledge verification. Our tool is narrower: it only checks whether sentences can be matched to provided sources, not whether those sources are factually correct.

**Testing and CI for ML systems:** The software engineering community has developed numerous testing frameworks for ML pipelines. `spanbind` extends pytest to enable lightweight grounding checks during RAG system development.

We note that this tool is not a substitute for human review, benchmark evaluation, or rigorous attribution research. It is a pragmatic CI utility.

## 5 Usage

Install: `pip install git+https://github.com/abinesha312/spanbind.git`. Demo: `spanbind demo`. pytest: `from spanbind import assert_bound; assert_bound(answer, sources)`. CLI: `spanbind check --answer answer.txt --source sources_dir/`.

## 6 Limitations and Future Work

**Current limitations:** The heuristic binder is not validated on attribution datasets; no precision/recall evaluation; threshold (0.55) is arbitrary; toy corpus (2 docs, 427 chars); cannot handle paraphrases or multi-hop reasoning.

**Future work:** Integration with published attribution models; benchmark evaluation (ASQA, AttributedQA); learned threshold tuning; multi-span support; improved number/entity handling.

**Use case:** Appropriate for catching obvious failures during development. *Not* for production safety or replacing human review.

## 7 Conclusion

`spanbind` provides a minimal, zero-dependency tool for detecting ungrounded claims in LLM outputs during continuous integration. The tool successfully binds sentences to source spans on a toy corpus and provides pytest integration for test-driven development of RAG systems. While the current implementation is limited to heuristic overlap matching on small examples, it demonstrates a practical approach to lightweight grounding verification in CI workflows.

The tool is released under the MIT license and is available at `https://github.com/abinesha312/spanbind`. All results in this paper are reproducible by running the repository's test suite and demo command.

## Broader Impact

This tool may help catch some grounding failures during development but produces false positives (paraphrases) and false negatives (coincidental overlap). Not suitable as a production safety mechanism. Human review remains essential.